

Spécifications (Cahier des charges)



TéléSPORT

Pages « Jeux Olympiques »

Contexte.....	2
Pages Ciblées.....	2
Périmètre.....	2
Hors périmètre.....	2
Détails des pages.....	3
1 - Dashboard (page d'accueil).....	3
2 - Page détail.....	4
User stories.....	5
Spécifications techniques.....	5
Modèle de données (simulé) et Services (données simulées).....	5
UI & composants attendus.....	6
Règles d'affichage.....	6
États, accessibilité & responsive.....	6
Contraintes non fonctionnelles.....	7
Critères d'acceptation (check-list).....	7

Contexte

TéléSport a préparé une section « Jeux Olympiques » pour visualiser les performances des pays. Ce projet contient deux pages principales en Angular :

1. un **Dashboard (page d'accueil)** : statistiques (nombre total des pays participants, nombre total de jeux olympiques), vue d'ensemble avec une pie chart,
2. une **Page détail pays** : statistiques (nombre total d'athlètes, nombre de participations, nombre de médailles) + évolution des médailles (graphique),

Les pages actuelles ne sont pas maintenables. Elles ne permettent pas les utilisations d'avoir une expérience fluide ni responsive.

Pages Ciblées

Les pages devront avoir une **navigation fluide** et une **UI responsive**, utilisables à la fois sur ordinateur, tablette et mobile.

Périmètre

- Graphiques (barres pour totaux ; ligne/aires pour évolution)
- Tri/sélection simple et navigation par clic
- Gestion des états : **loading**, **empty**, **error** (au minimum visuels)
- Accessibilité de base (contraste, focus visibles, ARIA simples)

Hors périmètre

- Authentification, rôles
- Édition/saisie des données
- Persistance réelle côté serveur

Détails des pages

1 - Dashboard (page d'accueil)

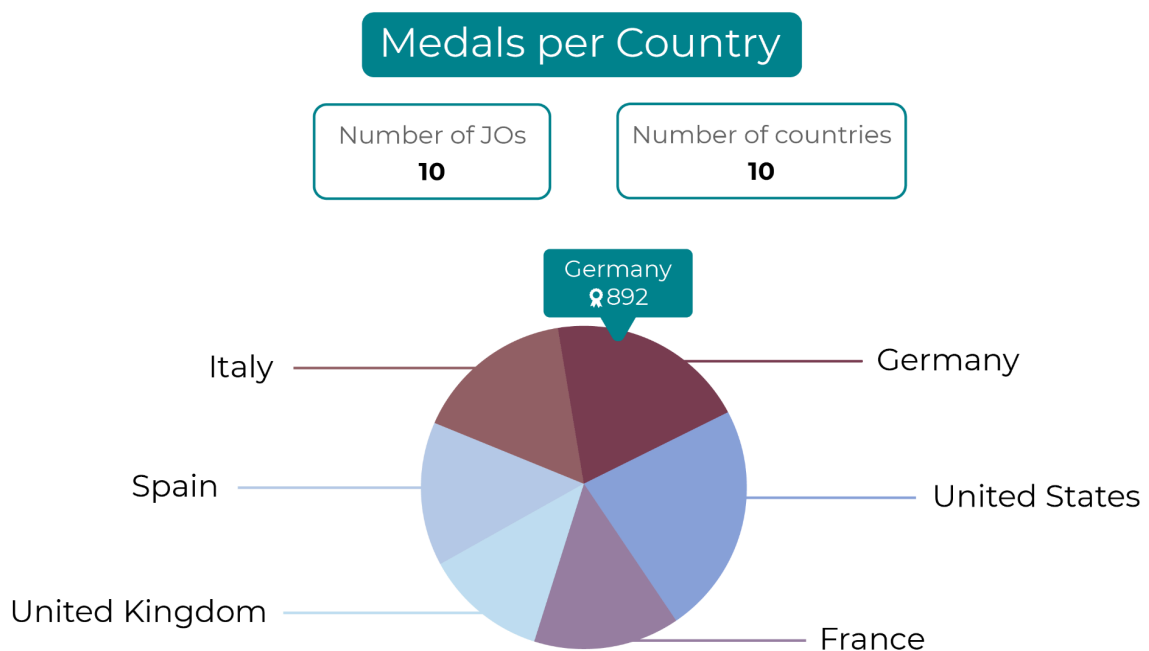
L'utilisateur arrivera sur cette page au démarrage de l'application (il s'agit de la route par défaut).

Cette page :

- (1) Présentera le contexte de l'application.
- (2) Contendra un graphique (type "bar" ou "pie", comme vous pouvez le voir dans le design sur Figma).

Ce graphique permettra la visualisation du nombre de médailles pour chaque pays, toutes années confondues. Pour pouvoir présenter ce graphique, l'affichage de la page d'accueil devra automatiquement déclencher la récupération des données nécessaires.

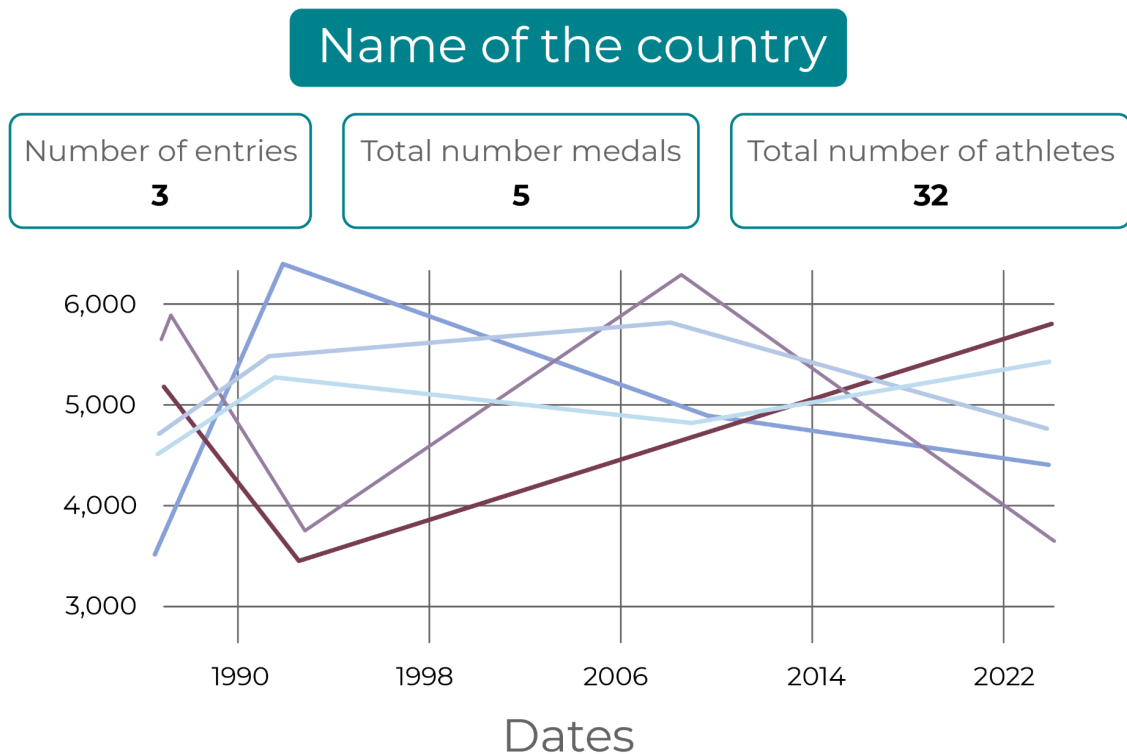
En cliquant sur un des pays, l'utilisateur sera redirigé vers une page "détail" à propos de ce pays.



NB : Pour ce projet, on ne prépare pas encore le détail par sport, cela fera partie d'une version ultérieure.

2 - Page détail

Lors de son affichage, cette page devra automatiquement récupérer les données propres au pays sélectionné par l'utilisateur.



Cette page devra ensuite afficher les données relatives à ce pays :

- nombre de participations aux JOs ;
- nombre total de médailles obtenues ;
- nombre total d'athlètes présentés aux JOs.

Pour finir, un nouveau graphique (de type “line” ou “bar”) devra permettre à l'utilisateur de visualiser sur cette page le nombre de médailles obtenues par le pays au cours de chaque édition des Jeux olympiques.

Il sera bien entendu essentiel de pouvoir retourner à la page d'accueil depuis cette page. Aussi, l'utilisateur peut saisir dans l'URL le pays dont il souhaite connaître les détails. Il faut donc vérifier que le pays existe

User stories

- **US-01** - En tant que visiteur, je veux voir un graphique des totaux de médailles par pays, afin de repérer les leaders.
- **US-02** - En tant que visiteur, je veux pouvoir cliquer sur un pays sur le Dashboard, afin d'ouvrir sa page de détail.
- **US-03** - En tant que visiteur, sur la route `/country/:id`, je veux voir les informations clés (participations, total médailles, athlètes) et un graphique d'évolution, afin de connaître les détails du pays sélectionné.
- **US-04** - En tant que visiteur, je veux pouvoir revenir au Dashboard depuis la page de détail, afin de naviguer facilement entre les pages.
- **US-05** - En tant que visiteur mobile, je veux que l'affichage reste lisible et utilisable, afin de consulter les informations sur mon appareil.

Spécifications techniques

Modèle de données (simulé) et Services (données simulées)

Créez des interfaces TypeScript dans `src/app/models/` :

```
TypeScript
export interface Participation {
  id: number,
  year: number,
  city: string,
  medalsCount: number,
  athleteCount: number
}

export interface Olympic {
  id: number,
  country: string,
  participations: Participation[]
}
```

Créer `DataService` dans `src/app/services/data.service.ts`

UI & composants attendus

HeaderComponent (composant réutilisable) :

- on constate que la page d'accueil et la page de détails d'un pays ont une structure d'en-tête similaire.
- on a toujours le titre de la page et l'affichage d'indicateurs propres à chaque page, un indicateur étant caractérisé par un libellé et une valeur.
- créer donc un composant **HeaderComponent** réutilisable qui affiche un titre et itère sur une liste d'indicateurs pour les afficher (pour chaque indicateur, le composant affiche le libellé et la valeur).

DashboardPage (page conteneur)

- utiliser le composant réutilisable **HeaderComponent** développé précédemment : titre de la page + KPIs (nombre de pays, nombre de JOs)
- ajouter un pie chart qui représente les pays

CountryDetailPage (page conteneur)

- utiliser un composant réutilisable **HeaderComponent** développé précédemment (titre de la page contenant le nom du pays + KPIs : participations, athlètes, total)
- une graphique ligne/aire par année

Règles d'affichage

- **Totals** = **gold** + **silver** + **bronze**
- Tri libre (alphabétique ou par total), mais **cohérent** Dashboard ↔ Détail
- Cliquer un élément du graphe Dashboard → **/country/:id**
- Bouton « Retour » sur la page détail (routerLink="/")

États, accessibilité & responsive

- **Loading** : squelettes simples (boîtes grises) ou spinner
- **Empty** : message « Aucune donnée »
- **Error** : message clair + bouton retour
- **Responsive** :
 - Desktop ≥ 1200px : 12 colonnes, contenus côte à côte
 - Tablette ~ 768-1199px : 8 colonnes, graphe pleine largeur
 - Mobile ≤ 767px : 4 colonnes, pile verticale
- **A11y** : contrastes AA, focus visibles, **aria-label** sur boutons/icônes, descriptions textuelles pour les graphes

Contraintes non fonctionnelles

- **Style** : cohérent, sobre (utilisez CSS/SCSS fournis par le starter ou Angular styles par défaut)
- **Code** : pas de `any`, factorisation des constantes, fichiers < 300 lignes idéalement
- **Git** : commits atomiques et messages clairs (feat, fix, refactor, docs)

Critères d'acceptation (check-list)

- `ng serve` OK sans erreur
- `/` affiche un **graphe fonctionnel** des pays et totaux
- Clic pays → `/country/:id`
- `/country/:id` affiche **KPI + évolution**
- Gestion d'ID invalide → message d'erreur/redirect
- **Responsive** testé (desktop/tablette/mobile)
- `DataService` centralise toutes les données (zéro tableau en dur dans les composants)
- **Types TS** définis (interfaces)
- `README.md` documente installation/structure/décisions
- `ARCHITECTURE.md` présent et compréhensible